Solution: Diagonal Traverse

Problem Statement

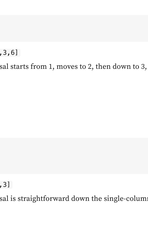
Given a 2D matrix of size `m x n`, return a 1D array containing elements of the matrix in the diagonal order.

Example 1:

- **Input:** `matrix =`

```
[[1, 2, 3, 4],
 [5, 6, 7, 8],
 [9, 10, 11, 12],
 [13, 14, 15, 16]]
```

- **Expected Output:** `[1, 2, 5, 9, 6, 3, 4, 7, 10, 13, 14, 11, 8, 12, 15, 16]`
- **Justification:** Traversal begins at 1, moves to 2, diagonally down to 5, and so on.



Example 2:

- **Input:** `matrix =`

```
[[1,5],
 [3,6]]
```

- **Expected Output:** `[1,5,3,6]`
- **Justification:** The traversal starts from 1, moves to 2, then down to 3, and finally to 4.

Example 3:

- **Input:** `matrix =`

```
[[1],
 [2],
 [3]]
```

- **Expected Output:** `[1,2,3]`
- **Justification:** The traversal is straightforward down the single-column matrix.

Solution

To solve this problem, we need to cleverly navigate the 2D array. The key is to recognize the pattern of movement: up-right and down-left diagonals, alternating after hitting the boundaries. We utilize two pointers to track our current position in the matrix and a boolean flag to indicate the current direction of movement. This approach ensures we cover all elements in the required order.

By dynamically altering our direction at the array's boundaries, we effectively traverse all elements in the zigzag pattern. This method is efficient as it requires just a single pass through the matrix, making it a time-efficient solution. Moreover, it's intuitive as it closely follows the pattern described in the problem statement.

Step-by-step Algorithm

- Initialize pointers for `row` and `column`, and a boolean `flag` for the direction of movement.
- Iterate through the matrix elements until all elements are traversed.
  - If moving upwards:
    - Move diagonally `up-right` by decrementing the row and incrementing the column.
    - Check if the `boundary` is reached. If so, change direction and adjust row/column accordingly.
  - If moving downwards:
    - Move diagonally `down-left` by incrementing the row and decrementing the column.
    - Check if the boundary is reached. If so, change direction and adjust row/column accordingly.
- Throughout the traversal, add the current element to the result list.
- Return the result list after complete traversal.

Algorithm Walkthrough

- Given Input: `matrix =`

```
[[1, 2, 3, 4],
 [5, 6, 7, 8],
 [9, 10, 11, 12],
 [13, 14, 15, 16]]
```

1. **Start**: `row = 0, column = 0, upwards = true, result = []`.
   - Add `1`. `result = [1]`.
2. **Change diagonal**: Hit boundary. Change direction. `row = 0, column = 1, upwards = false`.
   - Add `2`. `result = [1, 2]`.
3. **Move Down-Left**: Within bounds. `row = 1, column = 0`.
   - Add `5`. `result = [1, 2, 5]`.
4. **Change diagonal**: Hit boundary. Change direction. `row = 2, column = 0, upwards = true`.
   - Add `9`. `result = [1, 2, 5, 9]`.
5. **Move Up-Right**: Within bounds. `row = 1, column = 1`.
   - Add `6`. `result = [1, 2, 5, 9, 6]`.
6. **Change diagonal**: Hit boundary. Change direction. `row = 0, column = 2, upwards = false`.
   - Add `3`. `result = [1, 2, 5, 9, 6, 3]`.
7. **Move Down-Left**: Within bounds. `row = 0, column = 3`.
   - Add `4`. `result = [1, 2, 5, 9, 6, 3, 4]`.
8. **Move Down-Left**: Within bounds. `row = 1, column = 2`.
   - Add `7`. `result = [1, 2, 5, 9, 6, 3, 4, 7]`.
9. **Move Up-Right**: Within bounds. `row = 2, column = 1`.
   - Add `10`. `result = [1, 2, 5, 9, 6, 3, 4, 7, 10]`.
10. **Change diagonal**: Hit boundary. Change direction. `row = 3, column = 0, upwards = false`.
    - Add `13`. `result = [1, 2, 5, 9, 6, 3, 4, 7, 10, 13]`.
11. **Move Down-Left**: Within bounds. `row = 3, column = 1`.
    - Add `14`. `result = [1, 2, 5, 9, 6, 3, 4, 7, 10, 13, 14]`.
12. **Move Down-Left**: Within bounds. `row = 2, column = 2`.
    - Add `11`. `result = [1, 2, 5, 9, 6, 3, 4, 7, 10, 13, 14, 11]`.
13. **Change diagonal**: Hit boundary. Change direction. `row = 1, column = 3. upwards = false`.
    - Add `8`. `result = [1, 2, 5, 9, 6, 3, 4, 7, 10, 13, 14, 11, 8]`.
14. **Move Up-Right**: Within bounds. `row = 3, column = 2`.
    - Add `12`. `result = [1, 2, 5, 9, 6, 3, 4, 7, 10, 13, 14, 11, 8, 12]`.
15. **Change diagonal**: Hit boundary. Change direction. `row = 3, column = 2, upwards = true`.
    - Add `15`. `result = [1, 2, 5, 9, 6, 3, 4, 7, 10, 13, 14, 11, 8, 12, 15]`.
16. **Change diagonal**: Hit boundary. End of matrix. `row = 3, column = 3, upwards = false`.
    - Add `16`. Final `result = [1, 2, 5, 9, 6, 3, 4, 7, 10, 13, 14, 11, 8, 12, 15, 16]`.

The final output is `[1, 2, 5, 9, 6, 3, 4, 7, 10, 13, 14, 11, 8, 12, 15, 16]`.

Code

```java
import java.util.ArrayList;
import java.util.List;

public class Solution {
    public List<Integer> findDiagonalOrder(int[][] matrix) {
        List<Integer> result = new ArrayList<>();
        if (matrix == null || matrix.length == 0) return result;

        int N = matrix.length, M = matrix[0].length;
        int row = 0, column = 0;
        boolean upwards = true;

        while (row < N && column < M) {
            result.add(matrix[row][column]);

            // Determine the next cell in the traversal
            int newRow = upwards ? row - 1 : row + 1;
            int newColumn = upwards ? column + 1 : column - 1;

            // Check boundary conditions
            if (newRow < 0 || newRow == N || newColumn < 0 || newColumn == M) {
```

Run Code Result | Stdout

You must run your code first

Execute

Complexity Analysis

Time Complexity

The time complexity of the algorithm is `O(M x N)`, where **M** is the number of rows, and **N** is the number of columns in the matrix. The algorithm traverses each element once, so the time complexity is proportional to the total number of elements, which is `M x N`.

Space Complexity

The space used is for the output list, which holds all elements of the matrix. Since there are `M x N` elements, the space complexity is `O(M x N)`.